

Database Systems and Laboratory

SQL - Queries

Nicola Saccomanno

`nicola.saccomanno@uniud.it`

Bachelor degrees in Internet of things, Big Data, Machine Learning, & Computer Science
DMIF, University of Udine

A.Y. 2025/2026

Introduction

SQL (Structured Query Language) = DDL + DML

- ▶ The SQL language supports querying, definition, and data updating
- ▶ *Data Definition Language* (DDL)
 - ▶ Data
 - ▶ Integrity constraints
 - ▶ Views
 - ▶ Users management
 - ▶ Authorizations and permissions
- ▶ *Data Manipulation Language* (DML)
 - ▶ Inserts, updates, deletions
 - ▶ Queries
 - ▶ Transactions
 - ▶ Stored procedures and triggers
 - ▶ Embedding in OO or procedural languages
- ▶ Declarative fashion
- ▶ *Case insensitive*

SQL vs. Relational Algebra/Calculus

Two important elements of differentiation from relational algebra/calculus:

- (1) The results of projections are **multisets** rather than sets (by default, duplicates are not removed);
- (2) **Values** vs. relations (the result of the application of an aggregate function can, in specific contexts, be treated as a value and not as a relation).

Overview

The current SQL is the development of a query language for the relational DBMS **System R**, originally proposed at the IBM research laboratories in the latter half of the 1970s (SEQUEL - Structured English QUery Language).

Since the 1980s, it has been subject to intense **standardization** activities, mainly within the ANSI (American National Standards Institute) and ISO (the international body coordinating national bodies). The standardization process is continuous.

Advantages of standardization: **portability** and **uniformity** in accessing different databases.

Evolution of the SQL Standard - 1

The main stages of the SQL standardization process can be summarized as follows:

- (1) SQL-86 (Basic SQL), produced in 1986 by ANSI, contains all the basic constructs of the query language, but offers limited support for data definition and updating;
- (2) SQL-89 (Basic SQL), a limited extension of SQL-86, issued in 1989, introduces the concept of referential integrity;
- (3) SQL-92 (**SQL-2**), published in 1992, largely compatible with SQL-89, introduces many new features;

Evolution of the SQL Standard - 2

- (4) SQL:1999 (**SQL-3**), fully compatible with SQL-2, introduces object extensions, various new constructs, and new services such as triggers and recursive views;
- (5) SQL:2003, SQL:2006, and SQL:2011 introduce further object extensions, remove some unused constructs from previous standards, and include new parts such as SQL/JRT (integration of SQL with the Java language) and SQL/XML (XML data management);
- (6) SQL:2016, SQL:2019, and SQL:2023 adds support and operations useful to deal with more complex data formats, like property graph data or JSON for (multidimensional) arrays.

Unlike SQL-2, SQL-3 is organized into several appropriately numbered and named parts (for example, part 13 is related to SQL/JRT, part 14 to SQL/XML). Each part is produced by a specific technical committee and can be updated independently of the others.

Standard: Core and Extensions

Since SQL:1999, the standard is divided into a **core** and a set of **specialized extensions**.

All SQL-compliant DBMSs should provide the core; the extensions can be implemented as additional modules for specific applications, such as:

- ▶ data mining
- ▶ geographical and/or temporal data
- ▶ data warehousing
- ▶ OLAP (OnLine Analytical Processing)
- ▶ multimedia data
- ▶ ...

DBMS and SQL Standard

The various available DBMSs show small differences in the implementation of the SQL language, especially regarding the more innovative features.

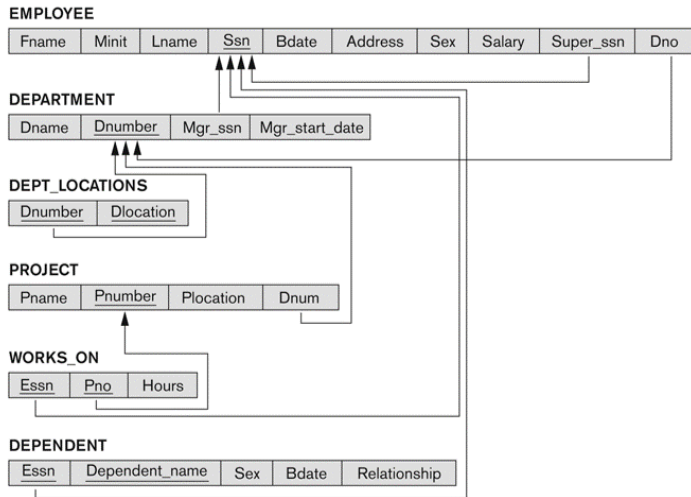
Uniformity is much greater regarding the more established functionalities.

Many systems provide **user-friendly interfaces** for querying, defining, and updating data (for example, visual interfaces).

However, all allow for the **explicit** (and complete) use of the SQL language.

Reference database schema

All the SQL examples presented in this slides will based on (a database built on) the following relational schema.



The SQL Query Language

An SQL query can contain up to **6 clauses**, of which only the first 2 are mandatory:

```
SELECT    ⟨ list of attributes ⟩ (target list)
FROM      ⟨ list of tables ⟩
WHERE     ⟨ condition ⟩
GROUP BY  ⟨ list of attributes (grouping) ⟩
HAVING    ⟨ condition (grouping) ⟩
ORDER BY  ⟨ list of attributes ⟩
```

The first 3 clauses constitute the **fundamental block** (also known as mapping).

Note that the order by which these 6 clauses are executed differs from that of writing:
FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY

The Fundamental Block

- SELECT** specifies the attributes and/or functions (generally, expressions) whose values are **returned by the evaluation** of the query (schema of the result relation). It can be specialized in various ways (e.g., using the DISTINCT keyword).
- FROM** specifies all the relations that need to be accessed to retrieve the information requested by the query (not those involved in potential nested queries).
- WHERE** specifies the conditions (Boolean expressions) for the **selection** of tuples from the relations indicated in the FROM clause (selection conditions and join conditions; in SQL-2, it's possible to move join conditions into the FROM clause).

Basic Queries

The **fundamental block**: SELECT-FROM-WHERE

Syntax:

```
SELECT   AttrExpr [[AS]ALIAS] {, AttrExpr [[AS]ALIAS]}  
FROM     Table [[AS]ALIAS] {, Table [[AS]ALIAS]}  
WHERE    Condition
```

In the simplest case, the execution of this block consists of the PROJECTION on the attributes specified in the SELECT clause of the result of the SELECTION of tuples belonging to the CARTESIAN PRODUCT of the relations specified in the FROM clause based on the condition specified in the WHERE clause.

Use of Aliases and the Asterisk

Example 1. Find the salary of employees whose last name is Bianco.

```
SELECT    SALARY AS SALARYBIANCO
FROM      EMPLOYEE
WHERE     LNAME = 'BIANCO'
```

→ Similar to renaming operator in RA.

Example 2. Retrieve all information related to employees whose last name is Bianco.

```
SELECT    *
FROM      EMPLOYEE
WHERE     LNAME = 'BIANCO'
```

Use of Expressions and Join Operations

Example 3. Find the monthly salary of employees whose last name is Bianco.

```
SELECT    SALARY / 12 AS MONTHLYSALARY
FROM      EMPLOYEE
WHERE     LNAME = 'BIANCO'
```

Example 4. For each employee, retrieve the name of the department they work for.

```
SELECT    SSN, DNAME
FROM      EMPLOYEE, DEPARTMENT
WHERE     DNO = DNUMBER
```

SELECT-FROM Subblock and Use of DISTINCT

DISTINCT clause, applied after the SELECT, filters out FULL DUPLICATE ROWS, i.e., returns only distinct (different) values in the result set.

Example 5. Retrieve the salary of each employee.

```
SELECT    SSN, SALARY  
FROM      EMPLOYEE
```

Example 6. Determine the different salary amounts of the company's employees.

```
SELECT    DISTINCT SALARY  
FROM      EMPLOYEE
```

Dot Notation and Ambiguous Attribute Names

Example 7. For each employee, identified by first and last name, retrieve the numbers of the projects they work on.

```
SELECT    EMPLOYEE.SSN, EMPLOYEE.FNAME,  
          EMPLOYEE.LNAME, W.PNUMBER  
FROM      EMPLOYEE, WORKS_ON AS W  
WHERE     SSN = ESSN
```

Example 8. Retrieve the name and date of birth of dependents for each employee.

```
SELECT    SSN, D.DEPENDENT_NAME, D.BDATE  
FROM      EMPLOYEE, DEPENDENT D  
WHERE     SSN = ESSN
```

Aliases and Renaming Operation

Example 9. Retrieve the first and last name of supervisors of employees working for department 10.

```
SELECT    S.FNAME, S.LNAME
FROM      EMPLOYEE AS E S
WHERE     E.DNO = 10 AND E.SUPER_SSN = S.SSN
```

Example 10. Find departments that have at least one common location.

```
SELECT    DISTINCT L1.DNUMBER, L2.DNUMBER
FROM      DEPT_LOCATIONS L1 L2
WHERE     L1.DLOCATION = L2.DLOCATION AND
          L1.DNUMBER < L2.DNUMBER
```

Note: DISTINCT operates at the tuple level (not single component) as SQL is tuple-oriented.

Boolean Expressions in the WHERE Clause

Example 11. Retrieve the first and last name of male employees earning more than 40,000 euros (assuming no homonyms).

```
SELECT    FNAME, LNAME
FROM      EMPLOYEE
WHERE     SEX = 'M' AND SALARY > 40000
```

Example 12. Determine the employees with the last name BIANCO who work for department 2 or department 3.

```
SELECT    SSN
FROM      EMPLOYEE
WHERE     LNAME = 'BIANCO' AND (DNO = 2 OR DNO = 3)
```

Set Operations - 1

The **set operations**: UNION (union), EXCEPT (set difference) and INTERSECT (intersection).

Duplicates are removed unless explicitly requested to be kept (use of the keyword ALL).

Example 13. Select the birth dates of all employees and their dependents.

```
SELECT    BDATE
FROM      EMPLOYEE
UNION
SELECT    BDATE
FROM      DEPENDENT
```

Set Operations - 2

If the attributes have different names, a union can still be performed, and the result usually uses the names of the first operand (thus, the names of the attributes of the result relation may vary depending on the order of operands).

Example 14. Select the names and surnames of all employees.

```
SELECT  FNAME  
FROM    EMPLOYEE  
UNION  
SELECT  LNAME  
FROM    EMPLOYEE
```

Set Operations - 3

Example 15. Select the names and surnames of all employees in Department number 10, keeping any duplicates.

```
SELECT    FNAME
FROM      EMPLOYEE
WHERE     DNO = 10
UNION ALL
SELECT    LNAME
FROM      EMPLOYEE
WHERE     DNO = 10
```

Set Operations - 4

Example 16. Select the last names of employees that are also first names (of some employee).

```
SELECT  LNAME
FROM    EMPLOYEE
INTERSECT
SELECT  FNAME
FROM    EMPLOYEE
```

Example 17. Select the first names of employees that are not also last names (of some employee).

```
SELECT  FNAME
FROM    EMPLOYEE
EXCEPT
SELECT  LNAME
FROM    EMPLOYEE
```

Nested Queries

Nested Queries: In the WHERE clause, SQL allows comparing a value (possibly the result of an expression evaluated on a single tuple) with the result of the execution of a nested SQL query, using standard comparison operators.

In the simplest case, this value is the **value of a specific attribute**.

How to compare a single value with a set of values (result of an evaluation of a query)?

- ▶ Use of the keywords ANY (equivalently SOME) and ALL.

Use of the Keywords ANY and ALL

How are nested queries evaluated? An evaluation for each tuple or combination of tuples of the query to which they belong.

The **keyword ANY**: the tuple satisfies the condition if the comparison (with the specified operator) between the value that the attribute/s assume on the tuple and at least one of the elements returned by the evaluation of the nested query is TRUE.

The **keyword ALL**: the tuple satisfies the condition if the comparison (with the specified operator) between the value that the attribute/s assume on the tuple and each of the elements returned by the evaluation of the nested query is TRUE.

Note: Domain compatibility between the attribute/s returned by the query and the attribute/s with which the comparison is made is required.

Examples of Using the Keyword ANY

Example 18. Select employees belonging to a department with a location in Pordenone.

```
SELECT      *
FROM        EMPLOYEE
WHERE       DNO = ANY (SELECT  DNUMBER
                        FROM    DEPT_LOCATIONS
                        WHERE   DLOCATION = 'PORDENONE')
```

Example 19. Select the social security number of employees who have the same last name as an employee working for department 10.

```
SELECT      SSN
FROM        EMPLOYEE
WHERE       LNAME = ANY ( SELECT  LNAME
                        FROM    EMPLOYEE
                        WHERE   DNO = 10)
```

Examples of Using the Keyword ALL

Example 20. Select all departments where no employee earns more than 80,000 euros.

```
SELECT    DNUMBER
FROM      DEPARTMENT
WHERE     DNUMBER <> ALL ( SELECT  DNO
                           FROM    EMPLOYEE
                           WHERE   SALARY > 80000)
```

Example 21. Select the employee(s) with the highest salary.

```
SELECT    SSN
FROM      EMPLOYEE
WHERE     SALARY >= ALL ( SELECT  SALARY
                           FROM    EMPLOYEE)
```

Alternative Solutions

Example 20'. Select all departments where no employee earns more than 80,000 euros.

```
SELECT    DNUMBER
FROM      DEPARTMENT
EXCEPT
SELECT    DNO
FROM      EMPLOYEE
WHERE     SALARY > 80000
```

Example 21'. Select the employee(s) with the highest salary.

```
SELECT    SSN
FROM      EMPLOYEE
EXCEPT
SELECT    E1.SSN
FROM      EMPLOYEE AS E1 E2
WHERE     E1.SALARY < E2.SALARY
```

The IN Operator

Note that if a nested query returns a single value, it is indifferent to use the keyword ANY or ALL (the keyword can also be omitted).

To check the inclusion (or exclusion) of an element/value in a set of elements/values returned by a nested query, the **IN** operator (equivalent to = ANY) and **NOT IN** (equivalent to <> ALL) can be used.

Example 18'. Select employees belonging to a department with a location in Pordenone.

```
SELECT      *
FROM        EMPLOYEE
WHERE       DNO IN (SELECT  DNUMBER
                     FROM    DEPT_LOCATIONS
                     WHERE   DLOCATION = 'PORDENONE')
```

Correlated and Non-Correlated Nested Queries

In all the nested queries seen so far, the nested query can be **executed only once** (before analyzing the tuples of the external query): the result of the evaluation of the nested query **does not depend** on the specific tuple examined by the external query (nested query is **not correlated** with the external query).

Sometimes, the nested query must be able to refer to the tuples of the tables declared in the external query: the result of the evaluation of the nested query **depends** on the specific tuple examined by the external query.

A mechanism for *parameter binding* is needed: attributes of tables declared in the outermost query must be able to be recalled by the innermost query.

Evaluation of Correlated Nested Queries

For each tuple of the outermost query:

- ▶ **First**, the nested query is evaluated,
- ▶ **Then**, the condition of the WHERE clause of the outermost query is evaluated (an evaluation that involves the result of the nested query).

This process can be repeated any number of times, equal to the number of queries nested within each other in a query (issues of **readability** of the queries).

Visibility Rules (and Use of Aliases)

Visibility rules (or scoping): the attributes of a table can only be used within the query in which the table is declared or in a nested query (at any level) within it.

Use of aliases: there can be ambiguities when there are multiple attributes with the same name, one that appears in a table declared in the external query, one that appears in a table declared in the nested query.

Disambiguation rule: an unqualified attribute refers to the table declared in the **innermost** (nested) query. To refer to attributes with the same name of tables declared in more outer queries, the use of **aliases** (seen as tuple variables) is necessary.

Violation of Visibility Rules

A query that **does not respect** SQL's visibility rules.

Example 22. Select employees who belong to the Research department or to a department that has a location in at least one city where the Research department has a location.

```
SELECT    SSN
FROM      EMPLOYEE
WHERE     DNO IN (SELECT D1.DNUMBER
                  FROM    DEPARTMENT AS D1,
                        DEPT_LOCATIONS AS L1
                  WHERE   D1.DNUMBER = L1.DNUMBER AND
                        D1.DNAME = 'RESEARCH') OR
          DNO IN (SELECT DNUMBER
                  FROM    DEPT_LOCATIONS AS D2
                  WHERE   L1.DLOCATION = D2.DLOCATION)
```

This solution is incorrect because the tuple variable **L1** is not visible in the second nested query. Find a correct solution.

Correlated Nested Queries and Use of Aliases

Example 23. Determine the first and last name of all employees who have a dependent of their gender with the same name.

```
SELECT    FNAME, LNAME
FROM      EMPLOYEE E
WHERE     SSN IN ( SELECT  ESSN
                   FROM    DEPENDENT
                   WHERE    E.FNAME = DEPENDANT_NAME AND
                           E.GENDER = GENDER)
```

Example 24. Select employees who earn a salary different from all other employees in their department.

```
SELECT    SSN
FROM      EMPLOYEE AS E
WHERE     SALARY NOT IN ( SELECT  SALARY
                           FROM    EMPLOYEE
                           WHERE    SSN <> E.SSN AND DNO = E.DNO)
```

The EXISTS Boolean Function

SQL introduces a Boolean function (**EXISTS function**) that checks whether the result of a correlated nested query is empty or not.

- ▶ Non-empty result relation: **EXISTS**
- ▶ Empty result relation: **NOT EXISTS**

Since we are not interested in the specific tuples returned by the nested query, but only in the presence/absence of tuples, the select clause of the nested query usually takes the form: **SELECT ***.

Examples of Using EXISTS and NOT EXISTS

Example 25. Select employees who do not have dependents.

```
SELECT    SSN
FROM      EMPLOYEE
WHERE     NOT EXISTS ( SELECT *
                        FROM   DEPENDENT
                        WHERE  SSN = ESSN)
```

Example 26. Return the first and last names of department managers who have at least one dependent.

```
SELECT    FNAME, LNAME
FROM      EMPLOYEE
WHERE     EXISTS ( SELECT *
                  FROM   DEPARTMENT
                  WHERE  SSN = MGR_SSN) AND
          EXISTS ( SELECT *
                  FROM   DEPENDENT
                  WHERE  SSN = ESSN)
```

INTERSECT and EXCEPT via EXISTS - 1

From relational calculus on tuples with range declarations (an advanced topic related with relational algebra that we did not discussed), it follows that set operators of intersection and difference, unlike the union operator, are not strictly necessary.

Let's see how they can be defined in SQL using the Boolean function EXISTS.

Given two relations $R(A, B)$ and $S(C, D)$:

Intersection ($R \cap S$)	Set Difference ($R - S$)
SELECT A, B	SELECT A, B
FROM R	FROM R
INTERSECT	EXCEPT
SELECT C, D	SELECT C, D
FROM S	FROM S

INTERSECT and EXCEPT via EXISTS - 2

Given $R(A, B)$ and $S(C, D)$, the two previous queries can be rewritten as follows:

Intersection

```
SELECT  A, B
FROM    R
WHERE   EXISTS ( SELECT *
                  FROM  S
                  WHERE  C = A AND D = B)
```

Set Difference

```
SELECT  A, B
FROM    R
WHERE   NOT EXISTS ( SELECT *
                     FROM  S
                     WHERE  C = A AND D = B)
```

The CONTAINS Operator

The original implementation of SQL (System R) included a CONTAINS operator that allowed determining whether one set was contained within another.

Example 27. Find employees who work on all projects controlled by department 10.

```
SELECT  SSN
FROM    EMPLOYEE
WHERE   ( SELECT PNO
          FROM    WORKS_ON
          WHERE SSN = ESSN)
        CONTAINS
        ( SELECT PNUMBER
          FROM    PROJECT
          WHERE DNUM = 10)
```

CONTAINS was later removed from SQL.

How to Express the CONTAINS Operator

Via NOT EXISTS and SET DIFFERENCE

Given two relations $R(A, B)$ and $S(C, D)$, let's determine if $S \subseteq R$.

Observe that $S \subseteq R \leftrightarrow S - R = \emptyset$:

- ▶ we can use one NOT EXISTS to check the emptiness
- ▶ and, the EXCEPT operator to compute the set difference

```
SELECT *  
FROM ...  
WHERE NOT EXISTS ( SELECT C, D  
                    FROM S  
                    EXCEPT  
                    SELECT A, B  
                    FROM R )
```

N.B.: We need another bit to make the query meaningful...

How to Express the CONTAINS Operator

Via NOT EXISTS and SET DIFFERENCE

Example 27'. Find employees who work on all projects controlled by department 10.

```
SELECT SSN
FROM EMPLOYEE AS E
WHERE NOT EXISTS ( SELECT PNUMBER
                   FROM PROJECT AS P
                   WHERE DNUM = 10
                   EXCEPT
                   SELECT PNO
                   FROM WORKS_ON
                   WHERE E.SSN = ESSN)
```

Note: Aliases are not needed here; they are used just for the sake of clarity.

How to Express the CONTAINS Operator

Via nested NOT EXISTS

Given two relations $R(A, B)$ and $S(C, D)$, let's determine if $S \subseteq R$.

Observe that $S \subseteq R \leftrightarrow S - R = \emptyset$:

- ▶ we can use one (the outermost) NOT EXISTS to check the emptiness
- ▶ and, another (the nested) NOT EXISTS to compute the set difference

```
SELECT *  
FROM ...  
WHERE NOT EXISTS ( SELECT *  
                    FROM   S  
                    WHERE   NOT EXISTS ( SELECT *  
                                        FROM   R  
                                        WHERE  C = A AND D = B))
```

N.B.: We need another bit to make the query meaningful...

How to Express the CONTAINS Operator

Via nested NOT EXISTS

Example 27'. Find employees who work on all projects controlled by department 10.

```
SELECT SSN
FROM EMPLOYEE AS E
WHERE NOT EXISTS ( SELECT *
                    FROM PROJECT AS P
                    WHERE DNUM = 10
                    AND
                    NOT EXISTS ( SELECT *
                                FROM WORKS_ON
                                WHERE E.SSN = ESSN AND
                                       P.PNUMBER = PNO)))
```

"Given a candidate employee, it is a good candidate if there does not exist a project controlled by department 10 that is not in the set of projects worked on by the candidate employee."

Note: Aliases are not needed here; they are used just for the sake of clarity.

Universal queries and NULL values

A toy table built for NULL semantics

Natural language query. Find all entities (E) that are associated with all values (V) associated with entity ROSSI.

Toy table.

$T(\text{ID}, \text{E}, \text{V})$

Example instance.

<u>ID</u>	E	V
1	ROSSI	P1
2	ROSSI	NULL
3	BIANCHI	P1
4	BIANCHI	NULL
5	VERDI	P1

ID is just a surrogate key. The universal query ranges over attribute V, which is intentionally allowed to contain NULL.

Universal queries and NULL values

Via nested NOT EXISTS

```
SELECT DISTINCT H.E
FROM   T AS H
WHERE NOT EXISTS ( SELECT *
                   FROM   T AS R
                   WHERE   R.E = 'ROSSI' AND
                           NOT EXISTS ( SELECT *
                                       FROM   T AS X
                                       WHERE  X.E = H.E AND X.V = R.V))
```

What goes wrong?

For the tuple of R (2, ROSSI, **NULL**), the inner condition becomes:

$$X.V = \mathbf{NULL}$$

which is never TRUE. Therefore the inner NOT EXISTS is TRUE, so (2,ROSSI,NULL) is treated as a counterexample for every candidate.

Consequence. Even BIANCHI is rejected, although it contains both visible values of ROSSI.

Critical point. With ordinary equality, a NULL in the universally quantified set behaves like an *unsatisfiable requirement*.

Result on this instance. No tuple is returned.

Universal queries and NULL values

Via NOT EXISTS and set difference

```
SELECT DISTINCT H.E
FROM   T AS H
WHERE  NOT EXISTS ( SELECT  R.V
                    FROM    T AS R
                    WHERE   R.E = 'ROSSI'
                    EXCEPT
                    SELECT  X.V
                    FROM    T AS X
                    WHERE   X.E = H.E)
```

What happens?

For BIANCHI, the set difference is:

$$\{P1, NULL\} - \{P1, NULL\} = \emptyset$$

Hence BIANCHI is returned.

For VERDI, the set difference is:

$$\{P1, NULL\} - \{P1\} = \{NULL\}$$

Hence VERDI is rejected.

Critical point. In EXCEPT, NULL is compared in a set-theoretic way: a NULL on the left is removed if a matching NULL is also present on the right.

Result on this instance. Only BIANCHI is returned.

Universal queries and NULL values

Using NOT IN instead of the inner NOT EXISTS

```
SELECT DISTINCT H.E
FROM   T AS H
WHERE  NOT EXISTS ( SELECT *
                    FROM   T AS R
                    WHERE  R.E = 'ROSSI' AND
                           R.V NOT IN ( SELECT X.V
                                       FROM   T AS X
                                       WHERE  X.E = H.E))
```

Example where it fails.

ID	E	V
1	ROSSI	P1
2	ROSSI	P2
3	BIANCHI	P1
4	BIANCHI	NULL

Why it is dangerous

- ▶ If the left-hand side is NULL, then NOT IN does not return TRUE:

`NULL NOT IN ('P1') = UNKNOWN`

- ▶ If the right-hand side contains a NULL, then a missing value may go undetected:

`'P2' NOT IN ('P1', NULL) = UNKNOWN`

E.g., for candidate BIANCHI, when checking $R.V = P2$, we get: `'P2' NOT IN ('P1', NULL)`
This is not TRUE; it is UNKNOWN. Thus, the missing value P2 is not detected as counterexample.

Critical point. With NOT IN, both a NULL on the left-hand side and a NULL inside the right-hand side set may turn the predicate into UNKNOWN instead of TRUE.

The UNIQUE Boolean Function

The **UNIQUE** Boolean function allows verifying that there are no duplicates in the result of a nested query.

Example 28. Find employees who do not have two or more dependents of the same gender.

```
SELECT SSN
FROM   EMPLOYEE
WHERE  UNIQUE ( SELECT  GENDER
                  FROM    DEPENDENT
                  WHERE   SSN = ESSN)
```

Join Operations

SQL-2 allows specifying a table that results from a join operation in the FROM clause (separating selection conditions, in the WHERE clause, from join conditions, in the FROM clause).

Types of joins:

- ▶ INNER JOIN (or simply JOIN)
- ▶ LEFT OUTER JOIN, RIGHT OUTER JOIN, FULL OUTER JOIN (the word OUTER can be omitted)
- ▶ NATURAL JOIN
- ▶ NATURAL LEFT/RIGHT/FULL OUTER JOIN

INNER JOIN

Example 29. Retrieve the first name, last name, and address of employees belonging to the Research department.

```
SELECT FNAME, LNAME, ADDRESS  
FROM   (EMPLOYEE JOIN DEPARTMENT  
        ON DNO = DNUMBER)  
WHERE  DNAME = 'RESEARCH'
```

Example 30. For each project based in Tolmezzo, return the project number, the controlling department, and the last name of the manager of that department.

```
SELECT PNUMBER, DNUMBER, LNAME  
FROM   ((PROJECT JOIN DEPARTMENT ON DNUM = DNUMBER)  
        JOIN EMPLOYEE ON MGR_SSN = SSN)  
WHERE  PLOCATION = 'TOLMEZZO'
```

OUTER JOIN

Example 31. Return the first and last name of each employee and their direct supervisor (if any).

```
SELECT E.FNAME AS EMPFIRSTNAME, E.LNAME AS EMPLASTNAME,  
       S.FNAME AS SUPFIRSTNAME, S.LNAME AS SUPLASTNAME  
FROM   (EMPLOYEE AS E LEFT OUTER JOIN  
        EMPLOYEE AS S ON E.SUPERVISOR = S.SSN)
```

Example 32. Return all departments with any projects they control.

```
SELECT PNUMBER, DNUMBER  
FROM   (PROJECT RIGHT OUTER JOIN DEPARTMENT  
        ON DNUM = DNUMBER)
```

Aggregate Functions in SQL - 1

SQL provides some built-in functions such as **COUNT**, **SUM**, **AVG**, **MAX**, and **MIN** (some functions for calculating variance, median, etc., were added in SQL-99).

Example 33. Determine the total sum of salaries paid by the company to its employees, the average salary, the highest salary, and the lowest salary.

```
SELECT SUM(SALARY), AVG(SALARY), MAX(SALARY), MIN(SALARY)
FROM   EMPLOYEE
```

In the previous example, all aggregate functions consider all values other than the NULL value of the SALARY attribute (the same effect would be obtained using the **ALL** keyword).

Aggregate Functions in SQL - 2

The COUNT operator allows counting the number of rows in a table (option *), the number of non-NULL distinct values taken by the attribute to which the function is applied (keyword **DISTINCT**), and the number of rows that have a non-NULL value on the attribute to which the function is applied (keyword **ALL**).

Example 34. Determine the number of employees in department 3.

```
SELECT COUNT(*)  
FROM   EMPLOYEE  
WHERE  DNO = 3
```

Example 35. Determine the number of different salary amounts among the company's employees.

```
SELECT COUNT(DISTINCT SALARY)  
FROM   EMPLOYEE
```

Inconsistent Queries (Mismatch)

Aggregate functions do not provide a selection mechanism. They are applied to a set (of values) and return a single value.

Consequently, the SELECT clause cannot contain both an attribute (or more), which generates a value for each selected tuple, and an aggregate function (or more), which returns a single value for the entire set of tuples.

An example of an **inconsistent query**:

```
SELECT FNAME, LNAME, MAX(SALARY)
FROM   EMPLOYEE, DEPT_LOCATIONS
WHERE  DNO = DNUMBER AND DLOCATION = 'TRIESTE'
```

Queries with Grouping

Aggregate functions and the **GROUP BY** clause, which allows partitioning the tuples of a relation.

Example 36. For each department, determine the sum of the salaries of its members.

```
SELECT    DNO, SUM(SALARY)
FROM      EMPLOYEE
GROUP BY DNO
```

With the GROUP BY clause, the SELECT clause can contain only aggregate functions and (a subset of) the grouping attributes.

Syntactically Incorrect Queries

Example 37. For each department, return the number of members and the manager (**incorrect solution**).

```
SELECT    DNO, COUNT(*), MGR_SSN
FROM      EMPLOYEE, DEPARTMENT
WHERE     DNO = DNUMBER
GROUP BY  DNO
```

Example 37'. For each department, return the number of members and the manager (**correct solution**).

```
SELECT    DNO, COUNT(*), MGR_SSN
FROM      EMPLOYEE, DEPARTMENT
WHERE     DNO = DNUMBER
GROUP BY  DNO, MGR_SSN
```

Predicates on Groups

The HAVING clause can be used to restrict to only those classes of the partition (induced by the attributes of the GROUP BY clause) that meet a certain condition.

Example 38. Select only those departments that spend more than 1,000,000 euros in salaries for their members (reporting this expense).

```
SELECT    DNO, SUM(SALARY) AS TOTALSALARY
FROM      EMPLOYEE
GROUP BY  DNO
HAVING    SUM(SALARY) > 1000000
```

WHERE Conditions vs. HAVING Conditions - 1

Conditions specified by the **WHERE** clause are evaluated **first** to narrow down the set of tuples; **then**, once the set of tuples is partitioned, the conditions specified by the **HAVING** clause are evaluated to eliminate some classes of the partition.

Example 39. For each department with more than 5 members, determine the number of members earning more than 60,000 euros (**incorrect solution: why?**).

```
SELECT    DNO, COUNT(*)
FROM      EMPLOYEE
WHERE     SALARY > 60000
GROUP BY DNO
HAVING    COUNT(*) > 5
```

WHERE Conditions vs. HAVING Conditions - 2

Example 39'. For each department with more than 5 members, determine the number of members earning more than 60,000 euros (**correct solution**).

```
SELECT    DNO, COUNT(*)
FROM      EMPLOYEE
WHERE     SALARY > 60000 AND
          DNO IN (SELECT    DNO
                   FROM      EMPLOYEE
                   GROUP BY DNO
                   HAVING    COUNT(*) > 5)

GROUP BY DNO
```

Relations as Values

A (sub)query that returns exactly one row and one column (singleton) can be treated as a single value.

Example 40. Determine the last name and first name of employees who have two or more dependents (using aggregate functions).

```
SELECT    LNAME, FNAME
FROM      EMPLOYEE
WHERE     (SELECT    COUNT(*)
          FROM      DEPENDENT
          WHERE      SSN = ESSN) >= 2
```

Miscellanea - 1

In comparisons with the result of nested queries, **tuples of attributes/values** can be used instead of single attributes/values.

Example 41. Select employees who dedicate the same number of hours to a project as the employee MNTGVN89S14J324Q.

```
SELECT    DISTINCT ESSN
FROM      WORKS_ON
WHERE     (PNO, HOURS) IN
           (SELECT    PNO, HOURS
            FROM      WORKS_ON
            WHERE     ESSN = 'MNTGVN89S14J324Q')
```

Miscellanea - 2

In the WHERE clause, sets of values can be introduced **explicitly**.

Example 42. Select employees who work on projects 1, 2, or 3.

```
SELECT    DISTINCT ESSN
FROM      WORKS_ON
WHERE     PNO IN {1, 2, 3}
```

Miscellanea - 3

String pattern matching: the **LIKE** operator.

Two reserved characters are used: %, which replaces any number (0 or more) of characters, and _, which replaces a single character.

Example 43. Select employees who reside in a (part of) city with Postal Code 33210.

```
SELECT    FNAME, LNAME
FROM      EMPLOYEE
WHERE     ADDRESS LIKE '%33210%'
```

Example 44. Select employees born in the sixties (assume BDATE format yyyy/mm/dd).

```
SELECT    FNAME, LNAME
FROM      EMPLOYEE
WHERE     BDATE LIKE '__ 6 _ _ _ _ _ _ _ _'
```

Miscellanea - 4

If % or _ need to be used as elements of a string, each occurrence must be preceded by an **escape character**, which is specified at the end of the string using the ESCAPE keyword.

- Examples: the sequence 'AB_CD\%EF'ESCAPE'\ ' represents the string 'AB_CD%EF' as the character \ is specified as the escape character.

For string data types, the **concatenation operator** || can be used to concatenate two strings.

In comparisons, the **BETWEEN operator** can be used, which allows expressing some conditions more compactly.

Example 45. Select employees of department 3 who earn between 30,000 and 40,000 euros (inclusive).

```
SELECT      *
FROM        EMPLOYEE
WHERE       (SALARY BETWEEN 30000 AND 40000) AND DNO=3
```

Miscellanea - 5

Tuples resulting from a query can be sorted, either in ascending or descending order, based on one or more of their attributes using the ORDER BY clause.

Example 46. Return the list of employees and the projects they work on, sorted by the department of affiliation (in descending order) and, within each department, in alphabetical order (ascending) by last name and first name.

```
SELECT    DNAME, LNAME, FNAME, PNAME
FROM      EMPLOYEE, DEPARTMENT, PROJECT, WORKS_ON
WHERE     DNO = DNUMBER AND SSN=ESSN AND
          PNO = PNUMBER
ORDER BY  DNAME DESC, LNAME ASC, FNAME ASC
```

Ascending order is the default (ASC can be omitted in the ORDER BY clause).

Miscellanea - 6

Handling null values: to select tuples that have (or do not have) a NULL value on a certain attribute, SQL provides the predicate **IS NULL** (or **IS NOT NULL**), which returns true if and only if the attribute has a null value (or does not have a null value).

Syntax: ATTRIBUTE IS [NOT] NULL

Example 47. Select all employees whose salary is not known.

```
SELECT    *  
FROM      EMPLOYEE  
WHERE     SALARY IS NULL
```

Note. SQL-89 uses two-valued logic; SQL-2 uses three-valued logic (true, false, unknown).

Exercise 1

Given the following relational schema related to films and actors:

- ▶ `FILM(CodeFilm, Title, Director, Year)`;
- ▶ `ACTORS(CodeActor, Name, Surname, Gender, DateOfBirth, Nationality)`;
- ▶ `INTERPRETATION(Film, Actor, Role)`.

It is assumed that each film is uniquely identified by a code and is characterized by its title, director, and release year. For simplicity, assume that each film is directed by a single director and each director is uniquely identified by their surname. There may be different films with the same title (this is the case, for example, with remakes). Each actor is uniquely identified by a code and is characterized by their name, surname, gender, date of birth, and nationality.

Assume that multiple actors can act in a given film and that an actor can act in multiple films. For simplicity, assume that, in each film, an actor can perform only one role.

Firstly define the primary keys, and the foreign keys of the given relations. Then, formulate appropriate queries in SQL to determine (using aggregate functions only if necessary):

1. Actors who have acted only in films by Kurosawa;
2. Actors who have acted in all films by Kurosawa;
3. Actors who have acted in all and only the films by Kurosawa.

Exercise 2

Given the following schema about a set of companies controlled by a given set of inspectors over a number of years:

- ▶ Inspector(SSN, Rank, YearOfBirth);
- ▶ Controlled(Inspector, Company, Year, Evaluation);
- ▶ LocatedIn(Company, City).

Assume that each inspector is identified by their tax code and is characterized by a rank, which describes their professional level, and their year of birth. Furthermore, assume that for every inspection carried out by an inspector, the controlled company, the year of the inspection, and the evaluation (positive or negative) are reported. A given company can be controlled by multiple inspectors and that an inspector can control the same company multiple times, but not during the same year. Finally, specify the city where each company is headquartered. Firstly define the primary keys, and the foreign keys of the given relations. Then, formulate appropriate queries in SQL to determine (using aggregate functions only if necessary):

1. The companies that have been controlled by a single inspector;
2. The inspectors who, in the year 2013, controlled (exactly) one or two companies;
3. The inspectors who have controlled exactly the same companies.

References

Material for these slides is largely based on:

- ▶ Prof. Angelo Montanari's slides about SQL

<https://users.dimi.uniud.it/~angelo.montanari/courses.php>